

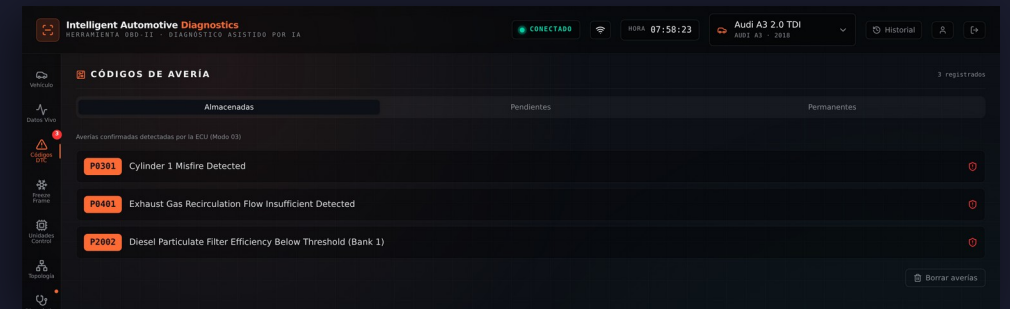
Intelligent Automotive Diagnostics

Diagnóstico OBD-II asistido por IA agéntica sobre el protocolo MCP

Jesús Ángel Novillo Lucas-Vaquero

Trabajo Fin de Máster · Máster en Desarrollo con IA · BIG school

LA HERRAMIENTA, EN FUNCIONAMIENTO



Diagnóstico de averías de vehículos asistido por IA

Se conecta al vehículo, interpreta los datos de sus centralitas y propone las causas de la avería.

Lo que aporta un escáner convencional

P0301 fallo de encendido en el cilindro 1

P0401 válvula EGR obstruida

P2002 filtro de partículas lleno

Hasta diez códigos. Registran el síntoma, no la causa que lo provoca.

Lo que añade esta herramienta

- Investiga por qué puede estar pasando eso
- Propone las causas posibles, no una sola
- Y los pasos a seguir para ir descartándolas
- Guarda el caso resuelto: sirve para el siguiente coche

Flujo de trabajo

Desde la conexión del adaptador hasta el registro del caso resuelto.



Diagnóstico determinista y diagnóstico cognitivo

El determinista siempre está disponible; el cognitivo requiere un modelo configurado.

Determinista

- Lee cuatro PIDs fijos: revoluciones, refrigerante, velocidad y admisión
- Más los DTCs y el freeze frame
- La criticidad sale de una regla: sin DTCs es baja, con freeze frame es crítica
- Solo necesita el coche, y responde al momento
- No aprende nada

Cognitivo

- El agente decide qué PIDs le hace falta leer
- Razona sobre lo que va encontrando, llamando a herramientas
- Necesita el coche, la API del modelo y los índices vectoriales
- Hasta 60 segundos de límite
- Y aprende: indexa PIDs, DTCs y el caso resuelto

Tecnologías utilizadas

TypeScript en todo el proyecto, y cada dependencia externa detrás de un puerto.

Lenguaje y runtime

TypeScript 5.7 estricto sobre Node 22

API

Express 5 · Zod · Helmet 8 · JWT · pino

Persistencia

SQLite con Drizzle ORM

Búsqueda vectorial

LanceDB · transformers.js en local

Agente

MCP SDK · SDK de Anthropic · SDK de OpenAI

Acceso al coche

ELM327 por serie o TCP · emulador Python

Interfaz

React 19 · Vite · TanStack Router y Query · Tailwind

Pruebas

Vitest · supertest · Playwright

Entrega

GitHub Actions · Docker · Caddy

Clean Architecture + Hexagonal

Tres capas —dominio, aplicación e infraestructura— con las dependencias siempre hacia dentro.

Qué ha aportado en este proyecto

- 01 El proyecto se ha desarrollado sin el vehículo**
- 02 La forma de conectarse al coche se decidió al final**
- 03 El diagnóstico cognitivo se añadió sin tocar el determinista**
- 04 Dos proveedores de modelo con el mismo código de diagnóstico**
- 05 Alta testabilidad: SAE J1979, ISO 15031 e ISO 3779 hay que probarlas con test**

Un ejemplo

Caso de uso de diagnóstico



Puerto de lectura del vehículo



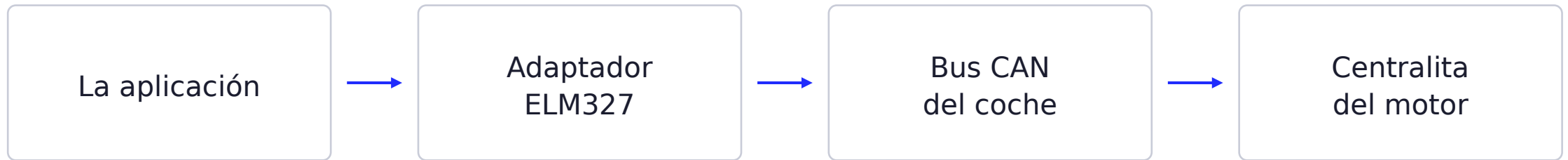
Lector ELM327 conectado al coche



Emulador en el portátil

Adquisición de datos mediante OBD-II

Ejemplo: lectura del régimen de giro del motor.



Va **01 0C** modo 01, PID 0C — revoluciones

Vuelve **41 0C 0B B8** los dos bytes de datos son A = 0B y B = B8

$$(A \times 256 + B) / 4 = (11 \times 256 + 184) / 4 = \mathbf{750 \text{ rpm}}$$

Persistencia: modelo relacional y modelo vectorial

Una responde por clave exacta; la otra, por similitud semántica.

SQLite

- Usuarios, talleres, vehículos, ECUs y sesiones
- Los catálogos de PID, DTC y ECU: los de la norma y los que se descubren
- Se consulta por clave: este VIN, este PID

LanceDB, vectorial

- Ese mismo conocimiento, indexado por significado
- Y los diagnósticos ya resueltos, que aquí no están
- Se consulta por parecido, no por clave
- El buscador de texto de SQLite no llega: «presión de aceite» no encuentra «oil pressure»

Base de datos relacional

13 tablas: todo aquello que se consulta por una clave conocida.

Qué guarda

- Usuarios: particulares o talleres, con su rol
- Vehículos por VIN y las ECUs de cada uno
- Sesiones de diagnóstico, con el informe congelado
- Lecturas de PID: el hexadecimal crudo y el valor ya convertido
- Logs y auditoría de cada petición
- Y crece solo: un PID leído que no esté en el catálogo se inserta con confianza 0,3

Lo que viene sembrado al arrancar

- 20** PIDs de la norma SAE J1979
- 23** códigos DTC estándar
- 27** fabricantes, por su código WMI

Base de datos vectorial e índice de confianza

Almacena el conocimiento que el sistema adquiere durante los diagnósticos.

Colecciones

- PIDs propietarios que no están en la norma, con su fórmula
- DTCs específicos de un fabricante
- Casos resueltos: síntomas, PIDs implicados y solución

Ejemplo de entrada indexada

embeddedText	"Audi A3 2.0 TDI con P0401 y P2002: EGR obstruida por carbonilla de un motor que consume aceite"
symptoms	ralentí inestable · pérdida de potencia
pidsInvolved	010C · 0105 · 22F40C
confidence	0,5 · manufacturer Audi · model A3

Índice de confianza

Se asigna al indexar, según la procedencia del dato. Es lo que pondera las búsquedas.

0,3 lo encontró el agente en internet

0,5 viene de un diagnóstico anterior

0,8 lo aportó el mecánico a mano

1,0 se ha leído del coche por OBD

Validar contra el coche sube la web a 0,7 y al mecánico a 0,9.

Model Context Protocol: herramientas del agente

El modelo no accede al vehículo: solicita herramientas y el sistema decide si se ejecutan.

Las 16 herramientas

Diagnóstico • 7

```
read_vin    read_pid    get_available_pids
get_dtc_codes  get_freeze_frame
get_vehicle_info  get_ecu_info
```

Conocimiento • 8

```
search_similar_pids  search_similar_dtcs
search_similar_diagnoses  search_similar_ecus
index_pid    index_dtc    index_diagnosis    index_ecu
```

Web • 1

```
web_search
```

Por qué MCP

- Es el estándar con el que un modelo pide herramientas, y solo sirve para eso
- El modelo solo actúa por ahí: no hay otra puerta al sistema
- Cada herramienta declara su esquema, y los argumentos se validan antes de ejecutarla
- El servidor vive en infraestructura: el modelo nunca ve el dominio
- Cambiar de modelo no toca las herramientas

Ciclo de razonamiento del agente

Del inicio del diagnóstico hasta la respuesta que recibe el mecánico.

Ciclo de ejecución

- 1 Al iniciar el diagnóstico se leen los PIDs generales, los DTCs y el freeze frame
- 2 Antes de llamar al modelo se buscan casos parecidos en la vectorial, y entran en el prompt
- 3 El modelo pide herramientas: mira el catálogo, lee más PIDs, busca lo que no conoce
- 4 Máximo 10 vueltas de ese ciclo, y 60 segundos de límite
- 5 Devuelve la explicación para el mecánico y un bloque JSON con severidad, confianza y recomendaciones
- 6 El caso resuelto se indexa con confianza 0,5, y la conversación sigue desde ahí

System prompt: 11 bloques

- Cómo explorar el coche y en qué orden
- Consultar el catálogo antes de inventarse nada
- Cómo aprender un PID, un DTC o una ECU nuevos
- Qué queda fuera de su ámbito y no debe contestar
- Que lo que llega de la web y del catálogo no es de fiar
- Cómo hablarle a un mecánico, y cómo rematar en JSON

Modelos empleados: lenguaje y embeddings

Uno razona y es intercambiable; el otro genera los vectores y se ejecuta en local.

El modelo de lenguaje

- Una variable de entorno: LLM_PROVIDER = anthropic u openai
- Los dos clientes implementan el mismo puerto, LlmClientPort
- El adaptador de OpenAI sirve también para DeepSeek, Groq, Mistral o xAI
- Sin proveedor configurado, el diagnóstico determinista sigue funcionando

El modelo de embeddings

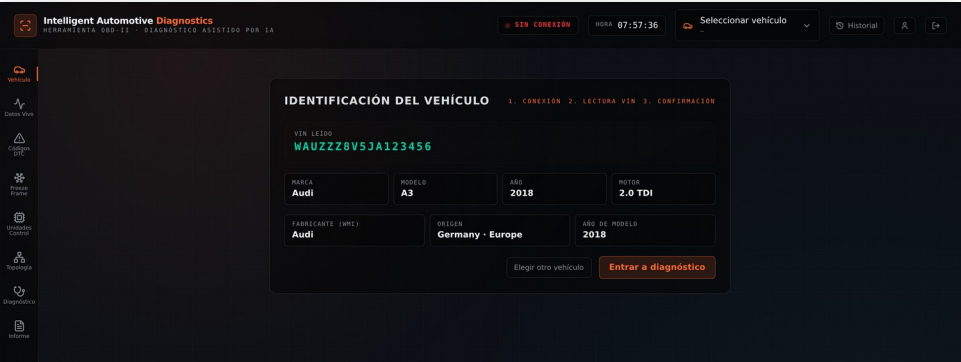
`paraphrase-multilingual-MiniLM-L12-v2`

- Corre en la propia máquina, con transformers.js
- Sin clave, sin red y sin coste por consulta
- Convierte cada texto en 384 números, normalizados
- Es multilingüe: por eso «presión de aceite» encuentra «oil pressure»

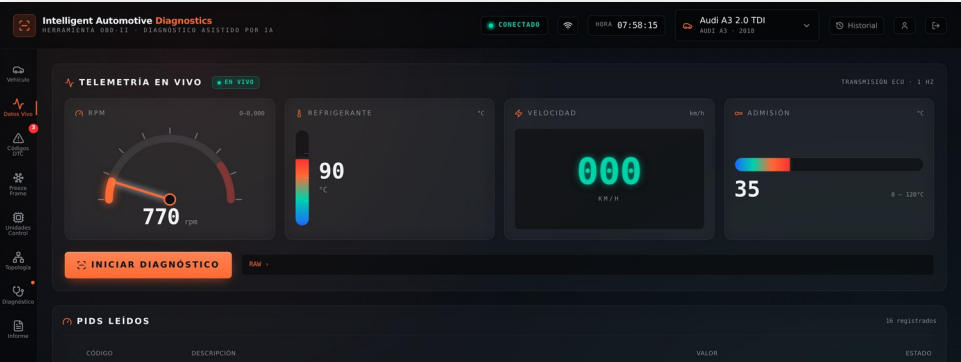
Resultados: la aplicación en funcionamiento

Audi A3 2.0 TDI. Todos los datos proceden del bus del vehículo.

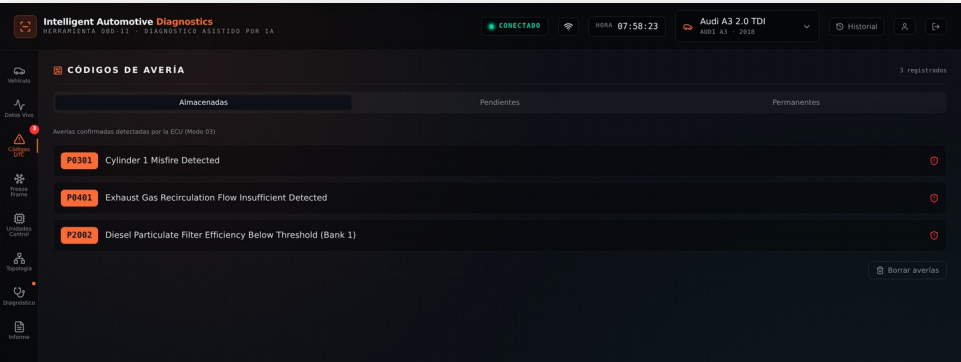
Lee el VIN del bus y saca marca, modelo y motor



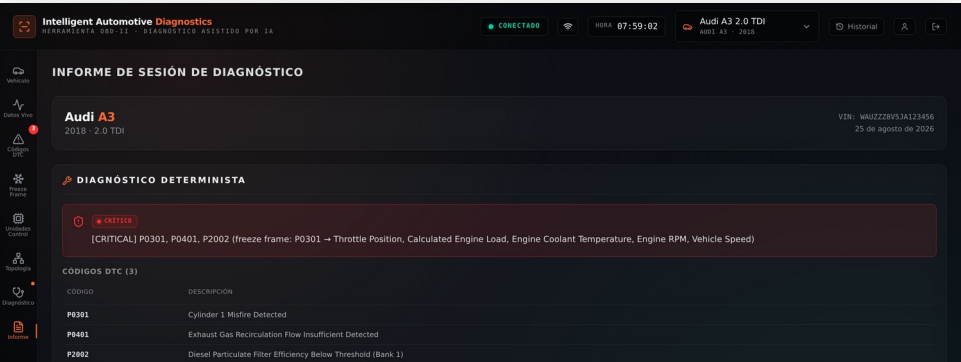
Telemetría en vivo y los PIDs que soporta



Las averías: modos 03, 07 y 0A



El informe que se congela con la sesión



Calidad del código y seguridad

Un único desarrollador: las comprobaciones automáticas son la red de seguridad.

Desarrollo

- TDD: primero el test que falla, luego el código
- 2.171 pruebas en verde, en 209 ficheros
- La cobertura se exige fichero a fichero, no de media
- 45 cambios especificados y archivados con OpenSpec

Verificaciones en cada push

- Lint, formato, tests, build y typecheck
- Las dos apps en paralelo, sobre Node 22
- Auditoría de dependencias que rompe si hay algo crítico
- OWASP API Top 10 2023: las diez categorías, documentadas

OWASP API Top 10 2023

El backend es una API REST, así que la lista que aplica es la de APIs.

API1	Broken Object Level Authorization	La sesión de otro usuario devuelve 404, no 403
API2	Broken Authentication	bcrypt 12 rondas, rotación de refresh y bloqueo a los 5 fallos
API3	Broken Object Property Level Authorization	Los esquemas Zod son la allowlist de campos
API4	Unrestricted Resource Consumption	login 5/min, diagnóstico 20/min, cognitivo 5/min
API5	Broken Function Level Authorization	Todo /api autenticado; administración tras requireAdmin
API6	Unrestricted Access to Sensitive Business Flows	Borrado de códigos con límite propio y apagable por entorno
API7	Server Side Request Forgery	El usuario no controla ninguna URL de salida
API8	Security Misconfiguration	Helmet 8: CSP, HSTS de un año y allowlist de CORS
API9	Improper Inventory Management	Especificación OpenAPI versionada, servida por la API
API10	Unsafe Consumption of APIs	Lo recuperado llega al modelo marcado como no fiable

La interfaz y los riesgos asumidos

La interfaz es una aplicación web, así que ahí manda el Top 10 de aplicaciones web.

Medidas en la interfaz

- React escapa el HTML por defecto, y no hay inyección directa de marcado sobre datos de usuario
- Cabecera CSP propia: default-src 'self', sin scripts de terceros
- Validación con Zod también en el cliente, en todos los formularios
- El token viaja en cabecera Bearer y no en cookie, así que no hay CSRF

Riesgos asumidos, no escondidos

- El token vive en localStorage: quedaría expuesto ante un XSS
- No hay doble factor de autenticación
- La base de datos no está cifrada en disco
- Los límites de peticiones viven en memoria y se pierden al reiniciar

Integración y despliegue continuos

Todo automatizado en GitHub Actions: integrar en main publica la nueva versión en producción.



Integración continua

- En cada push y en cada pull request
- Las dos aplicaciones en paralelo, sobre Node 22
- Lint, formato, pruebas, compilación y typecheck
- Auditoría de dependencias, que detiene la entrega ante una vulnerabilidad crítica

Despliegue continuo

- Se dispara al integrar en main
- Construye tres imágenes: API, interfaz y emulador
- Las publica en el registro de contenedores de GitHub
- El servidor las descarga y levanta la nueva versión, sin intervención manual

Conclusiones

Balance de haber desarrollado el proyecto con asistencia de IA.

Lo que aporta

- Se pueden construir cosas complejas mucho más rápido
- Lo que antes eran meses o años, ahora son semanas
- Y se aprende más por el camino: herramientas y frameworks que no habrías tocado

Lo que exige

- Acelera, pero también se equivoca
- Hay que revisar lo que se sube, no vale con fiarse
- No es todo tan bonito como parece: hay que dedicarle tiempo

Gracias por la atención

Jesús Ángel Novillo Lucas-Vaquero

Máster en Desarrollo con IA · BIG school

EL INFORME DE LA SESIÓN

Intelligent Automotive Diagnostics
HERRAMIENTA OBD-II · DIAGNÓSTICO ASISTIDO POR IA

CONECTADO · 07:59:02 · Audi A3 2.0 TDI · Audi A3 · 2018 · VIN: WAUZZZ8V53A123456 · 25 de agosto de 2026

INFORME DE SESIÓN DE DIAGNÓSTICO

Audi A3
2018 · 2.0 TDI

DIAGNÓSTICO DETERMINISTA

CRÍTICO
[CRITICAL] P0301, P0401, P2002 (freeze frame: P0301 → Throttle Position, Calculated Engine Load, Engine Coolant Temperature, Engine RPM, Vehicle Speed)

CÓDIGO	DESCRIPCIÓN
P0301	Cylinder 1 Misfire Detected
P0401	Exhaust Gas Recirculation Flow Insufficient Detected
P2002	Diesel Particulate Filter Efficiency Below Threshold (Bank 1)